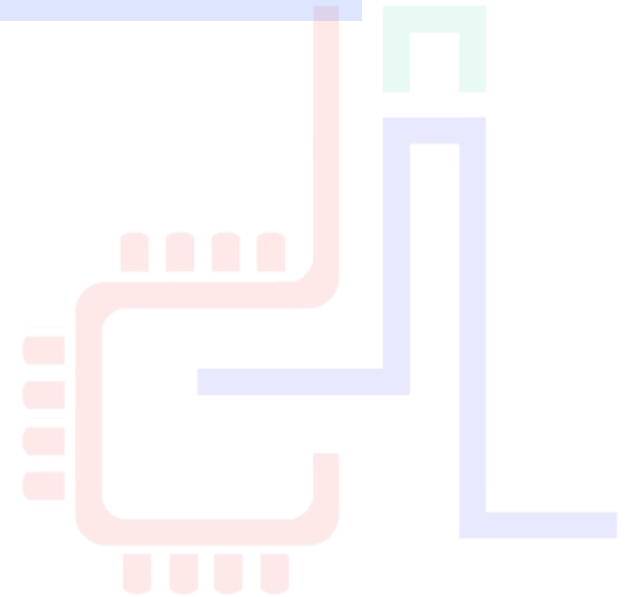


Framework BigDATA

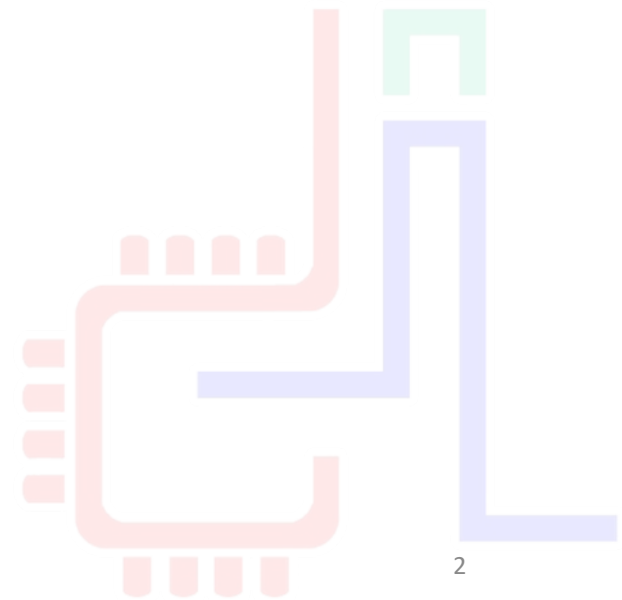
TAREK BEN MENA



CONTEXTE

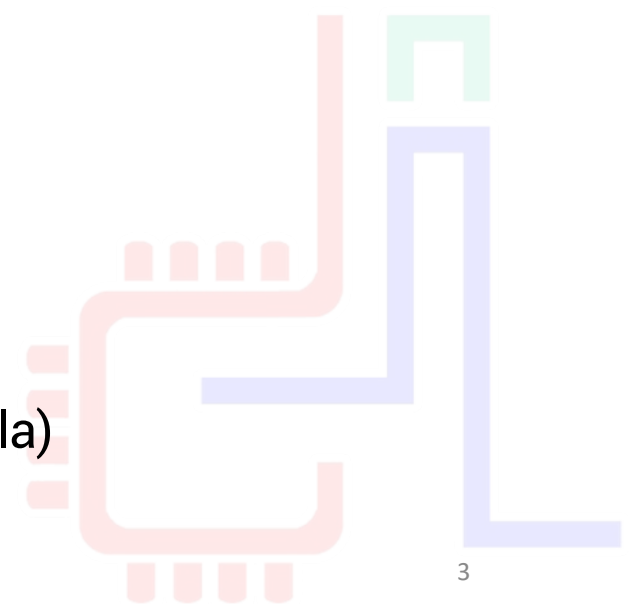
2K20

TBM



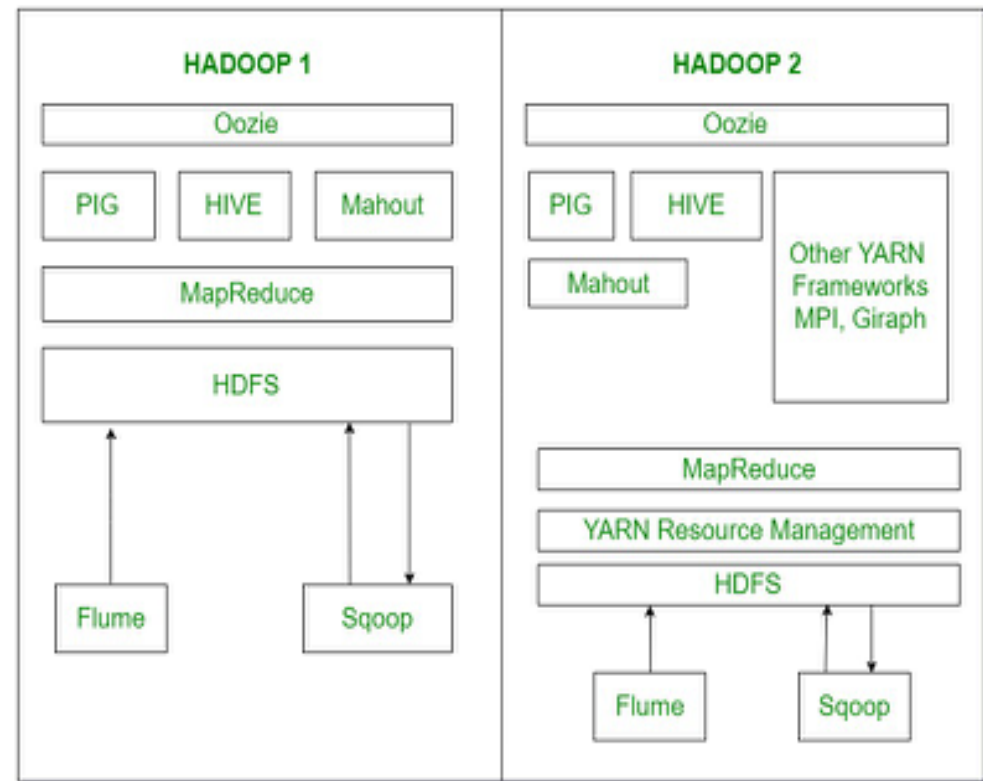
Contexte – le BigData

- Dans un contexte technologique où les données sont très faciles à produire, l'analyse devient de plus en plus complexe.
- Les 1^{ère} solutions à grosse volumétrie sont commerciales et se basent sur un modèle MPP ([Massively Parallel Processing](#)) ou shared nothing.
 - Exemple : Teradata (1983) , greenplum (2003) ...
- Solutions limitées aux grandes compagnies en raison du coût
- En 2003/2004, [Google](#) publie deux articles
 - MapReduce: Simplified Data Processing on Large Clusters
 - The Google File System
- Naissance de [Hadoop](#) en [2009](#) (Doug Cutting / Mike Cafarella)

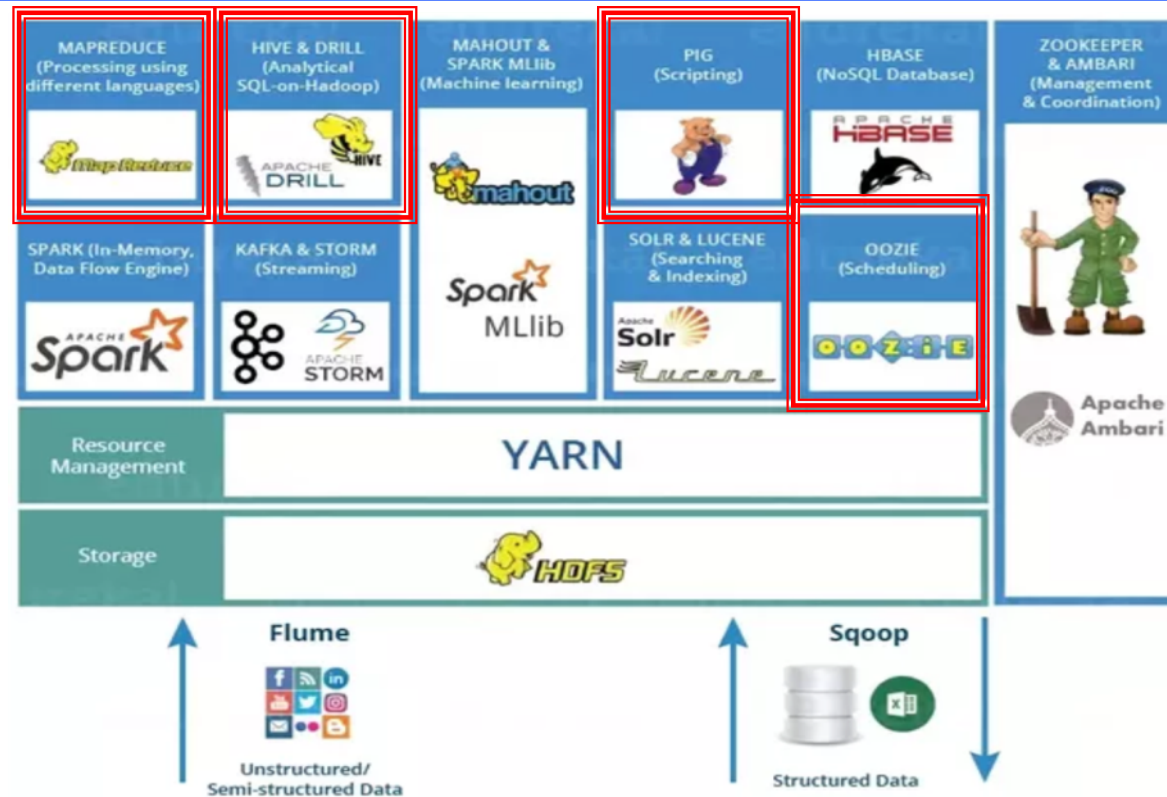


Context - Hadoop

- Hadoop est conçu sur plusieurs idées :
 - Développé en JAVA pour la **portabilité**
 - Traitement des données basé sur le **paradigme Map/R**
 - Le **même** matériel pour le **traitement et le stockage** des donnée
 - Les pannes matérielles sont **gérées au niveau logiciel**
 - Exécuter le code applicatif **au plus prêt** des données
 - Séparer la **logique du traitement** des données de la **logique de distribution** du calcul



Context - Hadoop



Pig: Langage de script (Pig Latin)

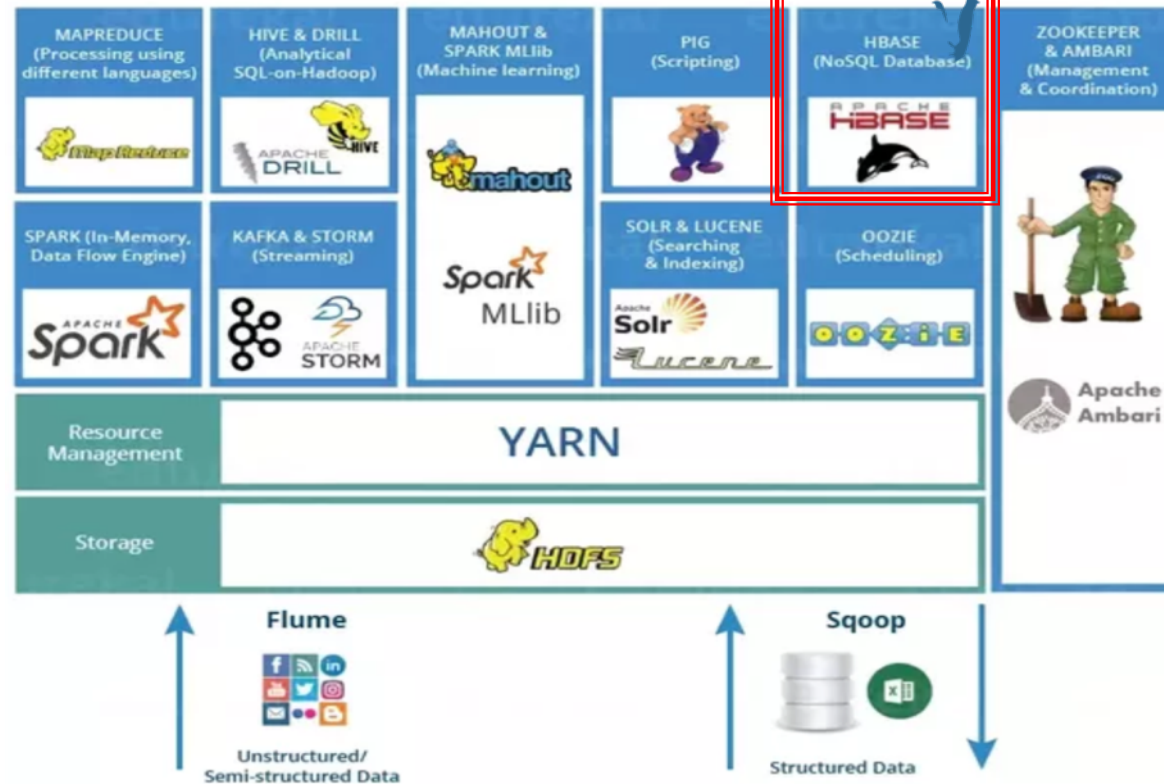
Hive: Langage proche de SQL (Hive QL)

R Connectors: Accès à HDFS, exécution de requêtes Map/Reduce à partir du langage R

Mahout: bibliothèque de machine learning

Oozie: ordonnancer les jobs Map Reduce avec des workflows

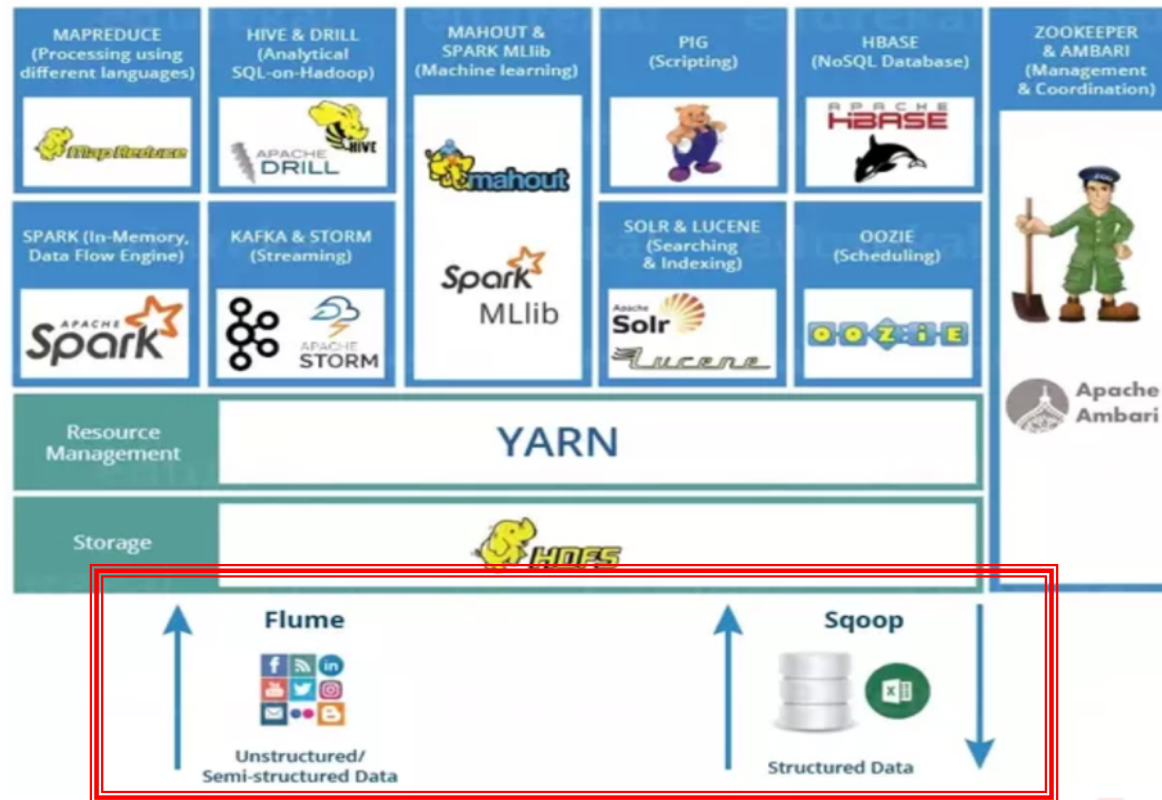
Context - Hadoop



Hbase : Base de données NoSQL orientée colonnes (table de +ieurs Milliard d'enregistrements avec des millions des colonnes)

Impala : permet le requêtage de données directement à partir de HDFS (ou de Hbase) en utilisant des requêtes Hive SQL

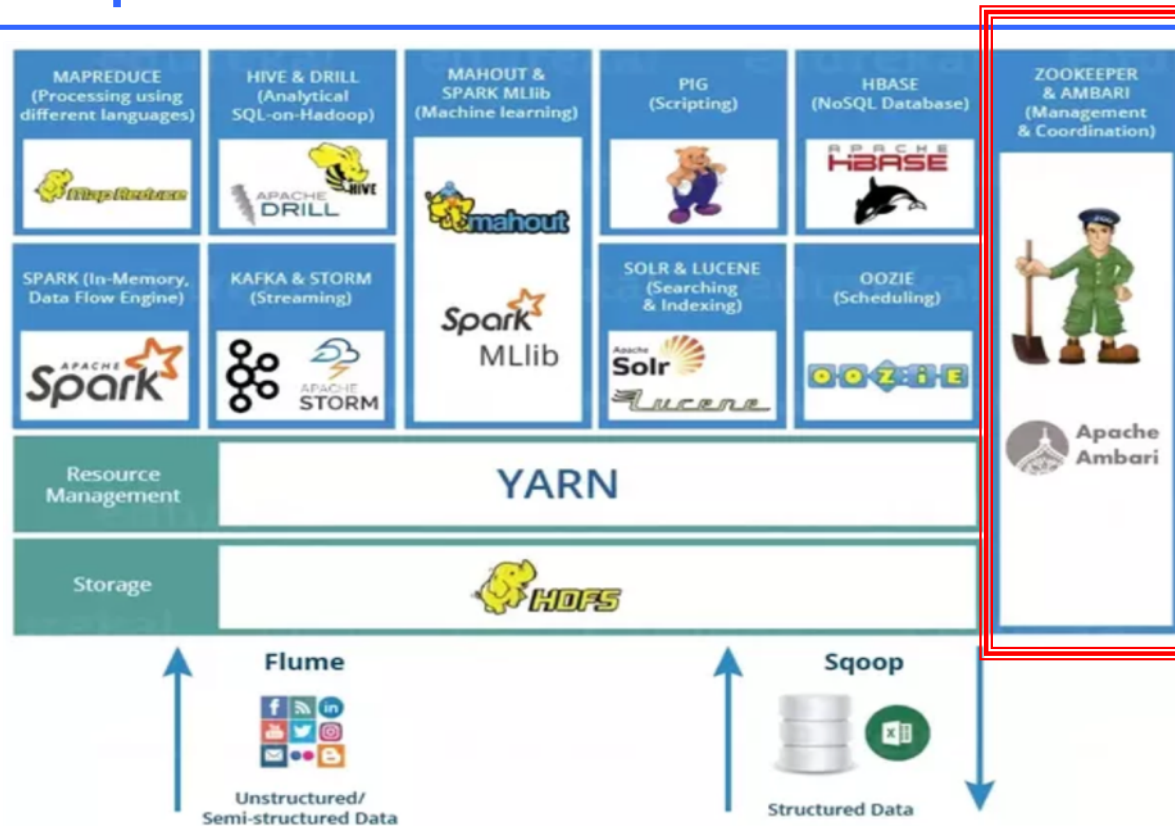
Context - Hadoop



Sqoop: Lecture et écriture des données à partir de bases de données externes

Flume: Collecte de logs et stockage dans HDFS

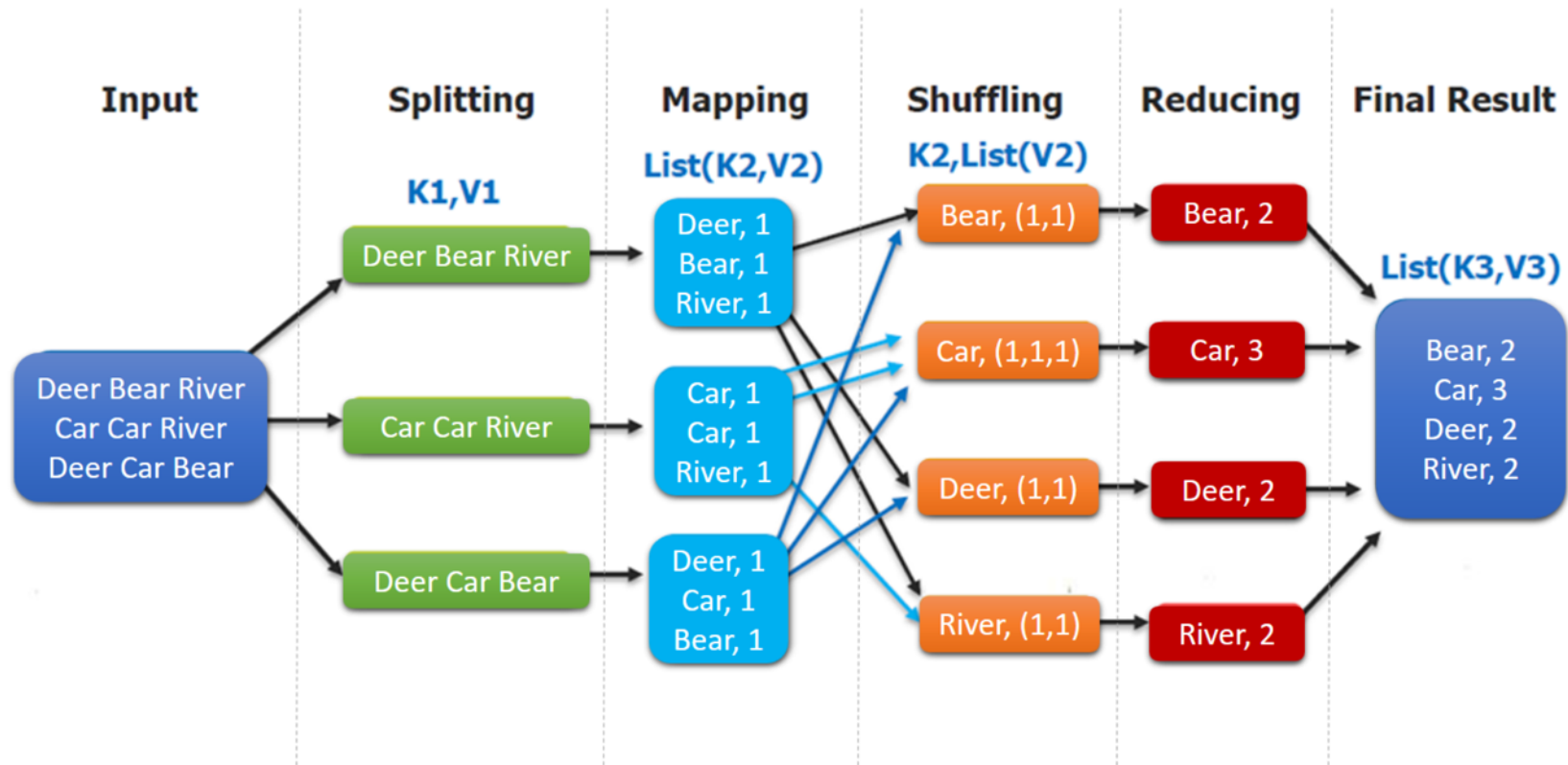
Context - Hadoop



Ambari: outil pour le provisionnement, gestion et monitoring des clusters

Zookeeper: (Gestionnaire de Configuration) fournit un service centralisé pour maintenir les informations de configuration, de nommage et de synchronisation distribuée

Rappel - MapReduce



MapReduce Word Count Process

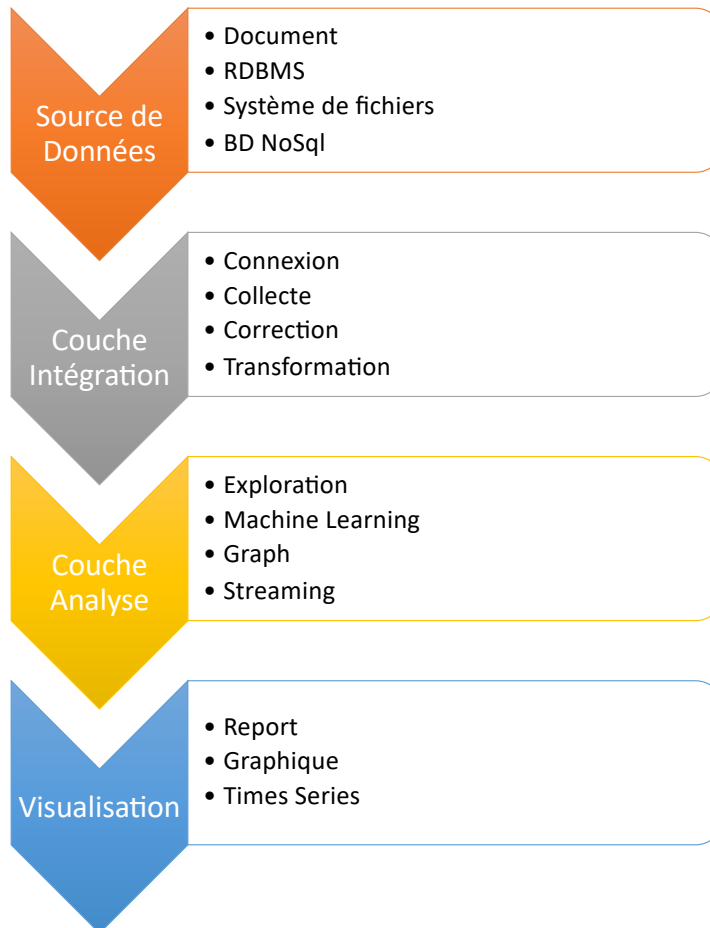
Rappel – MapReduce - Exemple

```
WordCount.java 83
1 package com.exemple;
2
3 import java.io.IOException;
4 import java.util.StringTokenizer;
5 import org.apache.hadoop.io.IntWritable;
6 import org.apache.hadoop.io.LongWritable;
7 import org.apache.hadoop.io.Text;
8 import org.apache.hadoop.mapreduce.Mapper;
9 import org.apache.hadoop.mapreduce.Reducer;
10 import org.apache.hadoop.conf.Configuration;
11 import org.apache.hadoop.mapreduce.Job;
12 import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
13 import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
14 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
15 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
16 import org.apache.hadoop.fs.Path;
17
18 public class WordCount{
19     public static class Map extends Mapper<LongWritable,Text,Text,IntWritable>{
20         public void map(LongWritable key, Text value,Context context) throw
21             String line = value.toString();
22             StringTokenizer tokenizer = new StringTokenizer(line);
23             while (tokenizer.hasMoreTokens()) {
24                 value.set(tokenizer.nextToken());
25                 context.write(value, new IntWritable(1));
26             }
27         }
28     }
29     public static class Reduce extends Reducer<Text,IntWritable,Text,IntWritable>{
30         public void reduce(Text key, Iterable<IntWritable> values,Context context) {
31             int sum=0;
32             for(IntWritable x: values)
33             {
34                 sum+=x.get();
35             }
36             context.write(key, new IntWritable(sum));
37         }
38     }
39
40     public static void main(String[] args) throws Exception {
41         Configuration conf= new Configuration();
42         Job job = new Job(conf,"My Word Count Program");
43         job.setJarByClass(WordCount.class);
44         job.setMapperClass(Map.class);
45         job.setReducerClass(Reduce.class);
46         job.setOutputKeyClass(Text.class);
47         job.setOutputValueClass(IntWritable.class);
48         job.setInputFormatClass(TextInputFormat.class);
49         job.setOutputFormatClass(TextOutputFormat.class);
50         Path outputPath = new Path(args[1]);
51         //Configuring the input/output path from the filesystem into the job
52         FileInputFormat.addInputPath(job, new Path(args[0]));
53         FileOutputFormat.setOutputPath(job, new Path(args[1]));
54         //deleting the output path automatically from hdfs so that we don't
55         outputPath.getFileSystem(conf).delete(outputPath);
56         //exiting the job only if the flag value becomes false
57         System.exit(job.waitForCompletion(true) ? 0 : 1);
58     }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
```

Les limites de MapReduce

- La spécification de l'itération reste à la charge du programmeur;
 - Il faut stocker le résultat d'un 1^{er} job dans une collection intermédiaire
 - puis réitérer le job en prenant la collection intermédiaire comme source.
- C'est laborieux pour l'implantation, et surtout très peu efficace quand la collection intermédiaire est grande.
- Le processus de **sérialisation/dé-sérialisation** sur disque propre à la gestion de la reprise sur panne en MapReduce entraîne des **performances médiocres**.
- Dans **Spark**, la méthode consiste à placer ces jeux de **données en mémoire RAM** et à éviter la pénalité des écritures sur le disque
 - Le défi est alors bien sûr de proposer une reprise sur panne automatique efficace

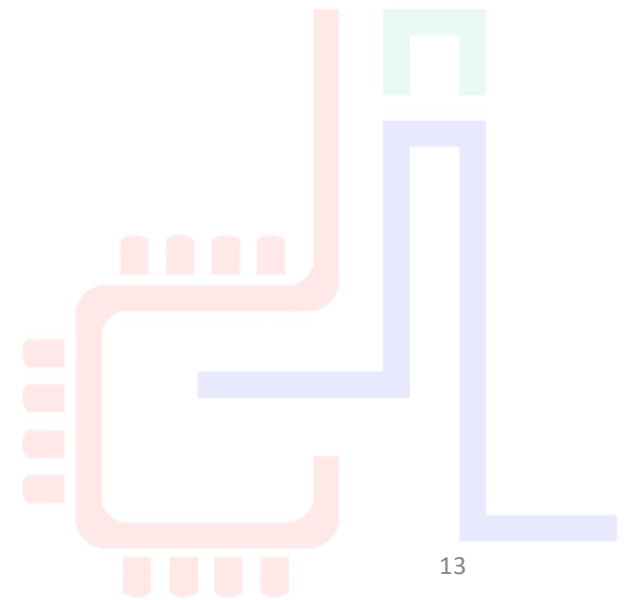
Traitement des données



C'EST QUOI SPARK?

2K20

TBM



Spark ? (1/2)

- SPARK s'inspire fortement de Hadoop
 - L'utilisation de matériel commun pour le traitement mais ne gère pas le stockage des données
 - Les pannes matérielles sont gérées au niveau logiciel (RDD)
 - Exécuter le code applicatif **au plus près des données**
 - Séparer la logique du **traitement des données** de la logique de **distribution du calcul**
 - Intègre son propre cluster manager **SPARK standalone** lequel est interchangeable avec **YARN** / **MESOS**
- C'est un **moteur de traitement parallèle** de données open source permettant d'effectuer des analyses rapides (**In-memory**) dédié au Big Data de manière distribuée (**cluster computing**).

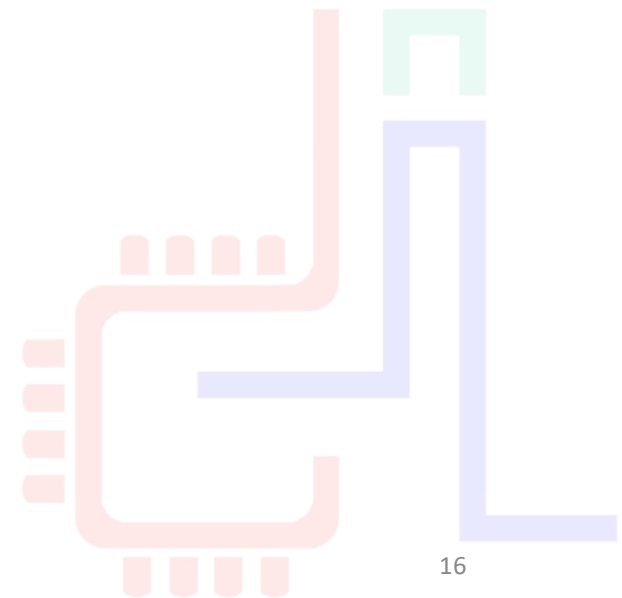
Spark ? (2/2)

- Framework est en passe de remplacer Hadoop.
 - **100 x** plus rapidement que Hadoop MapReduce in-memory,
 - **10 x** plus vite sur disque
- Spark permet de traiter des données issues de
 - Hadoop Distributed File System (HDFS),
 - les bases de données NoSQL,
 - ou les data stores de données relationnels comme Apache Hive.
- Développé en **SCALA**, langage basé sur une JVM.
 - 100 lignes de code JAVA peut être traduit en une dizaine de lignes en SCALA
 - Propose une API **Java** et **Python**
- Hérite des avantages de la **programmation fonctionnelle**
 - Le calcul est évalué comme une fonction mathématique
 - Le développement **d'applications parallélisées** est plus **facile**



Spark : Spécification

- Un **moteur d'exécution** basé sur des **opérateurs** de haut niveau, comme Pig. Comprend des opérateurs Map/Reduce, et d'autres opérateurs de second ordre
- Introduit un **concept de collection résidente en mémoire (RDD)** qui améliore considérablement certains traitements, dont ceux basés sur des itérations (PageRank, kMeans)
- **Fonctionnalités**
 - Batch / MapReduce
 - **MLlib** Machine Learning : la fouille de données
 - **Streaming** : le traitement de flux
 - **Spark SQL** : exécution des requêtes en langages SQL
 - **GraphX** : traitement des graphes



Spark : Fonctionnalités

- MLlib



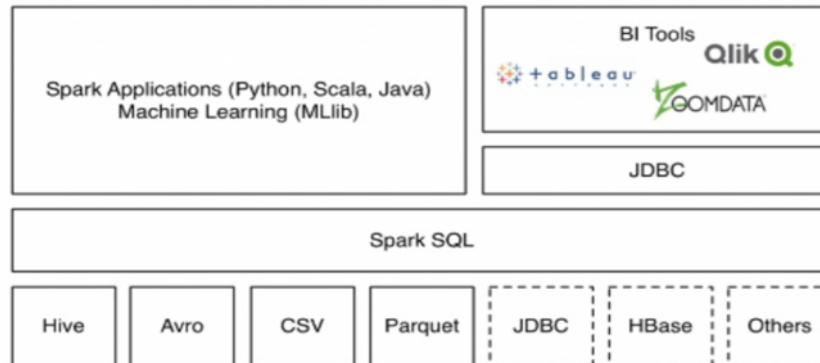
- Spark Stream



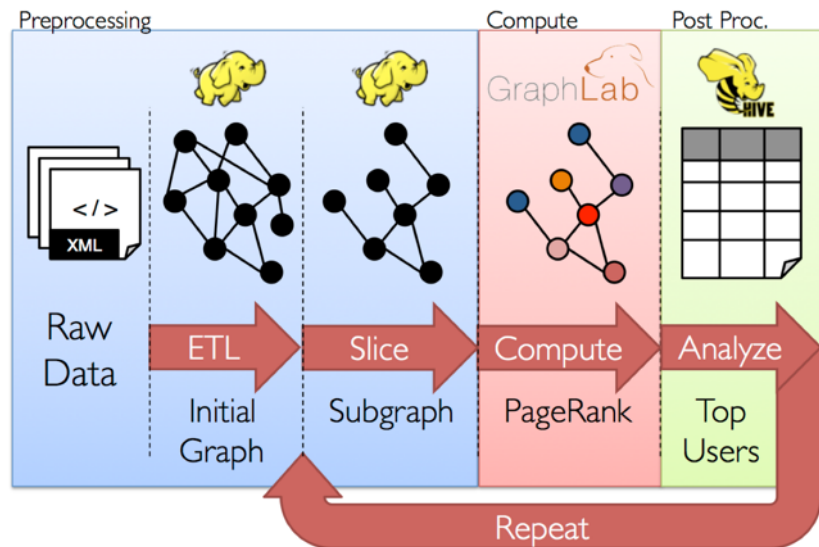
Spark : Fonctionnalités

- Spark SQL

Data source API



- GraphX



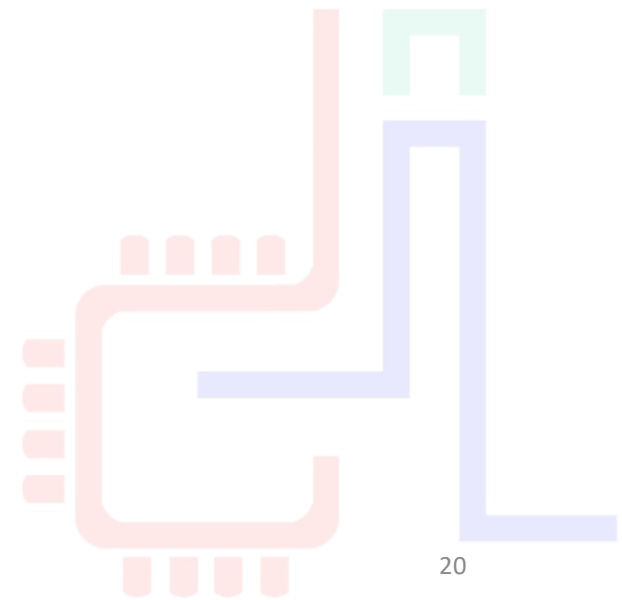
Spark : Historique

- **2009** Initialement développé par l'AMPLab de l'université de Californie à Berkeley par **Matei Zaharia**
- Transmis à la fondation Apache, développement open-source depuis **2013**
- En **2014**, Spark a gagné le Daytona GraySort Contest7 dont l'objectif est de trier **100 To** de données le plus rapidement possible. Ce record était préalablement détenu par Hadoop.
 - Spark (206 Machine, 23 min) vs Hadoop (2100 machines, 72 min)
- **2015** plus de 1000 collaborateurs, dont Intel, Facebook, IBM, Netflix
- Version actuelle : 3.0 . La version 2.0 a apporté des changements notables concernant les structures de données utilisées.
- Spark est utilisable avec plusieurs langages de programmation : Scala (natif), Java, Python, R, SQL.

INSTALLATION

2K20

TBM



20

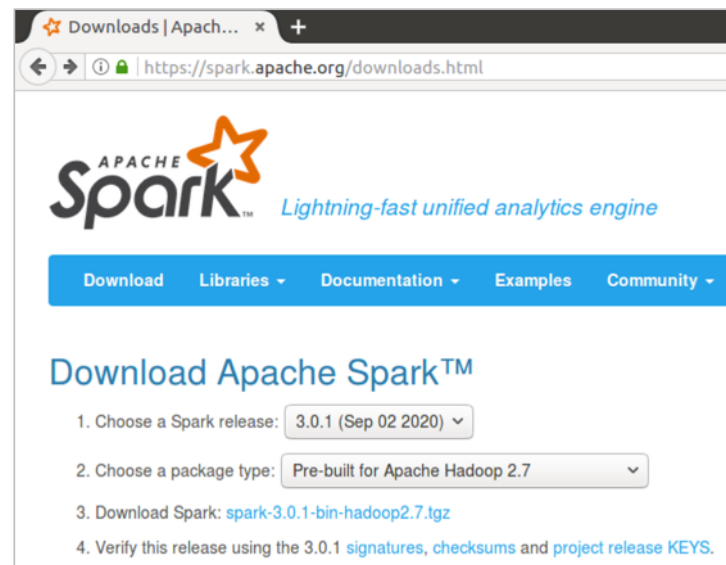
Installation Spark 3.0.1 sur Ubuntu 16.04

- Java est la seule dépendance pour installer Apache Spark

```
~$ sudo apt-get install default-jdk
```

```
spark@spark-VirtualBox:~$ java -version
openjdk version "1.8.0_265"
OpenJDK Runtime Environment (build 1.8.0_265-8u265-b01-0ubuntu2~16.04-b01)
OpenJDK Server VM (build 25.265-b01, mixed mode)
```

- Télécharger Spark
 - <https://www.apache.org/dyn/closer.lua/spark/spark-3.0.1/spark-3.0.1-bin-hadoop2.7.tgz>



Installation Spark 3.0.1 sur Ubuntu 16.04

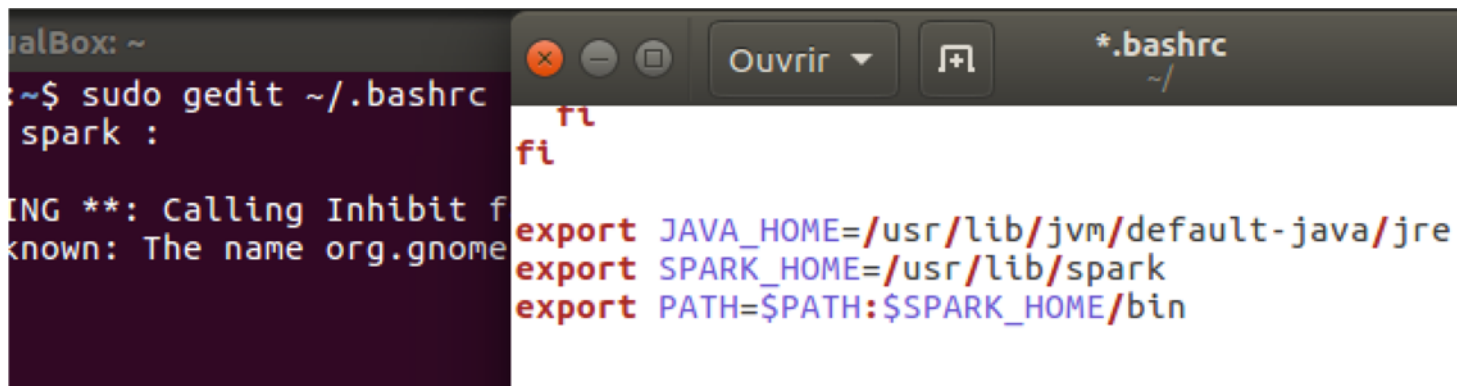
- Désarchiver le `spark-3.0.1-bin-hadoop2.7.tgz`

```
~$ tar xzvf ./Téléchargements/spark-3.0.1-bin-hadoop2.7.tgz
```

- Renommer le dossier `spark` et déplacer le sous `/usr/lib/`

```
~$ mv spark-3.0.1-bin-hadoop2.7/ spark
~$ sudo mv spark/ /usr/lib/
```

- Ajouter les variables d'environnement au `$PATH` à la fin du fichier `~/.bashrc`



The screenshot shows a terminal window on the left and a gedit editor window on the right. The terminal window displays the command `sudo gedit ~/.bashrc` and the output `spark :`. The gedit editor window shows the contents of the `~/.bashrc` file, which includes the following lines:

```
fi
fi
export JAVA_HOME=/usr/lib/jvm/default-java/jre
export SPARK_HOME=/usr/lib/spark
export PATH=$PATH:$SPARK_HOME/bin
```

Installation Spark 3.0.1 sur Ubuntu 16.04

- Relancer le terminal pour la prise en compte des nouvelle variable et taper

```
~$ spark-shell
```

```
spark@spark-VirtualBox: ~
20/10/05 09:33:36 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
20/10/05 09:33:37 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1601886831982
).
Spark session available as 'spark'.
Welcome to

  ____      _
 / ___|    / \
| |  | |  / _ \
| |  | | / ___ \
| |  | || |___| \
| |  | || |___| \
| |  | || |___| \
|_|  |_| \_____/

 version 3.0.1

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Lancement de Spark

Quitter l'ancien shell

spark context pour ce shell agit comme un master sur le local node avec 4 threads

```
scala> :quit
```

spark@spark-VirtualBox:~\$ spark-shell --master "local[4]"

20/10/05 10:16:23 WARN Utils: Your hostname, spark-VirtualBox resolves to a local IP address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)

20/10/05 10:16:23 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another IP address

20/10/05 10:16:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties

Setting default log level to "WARN".

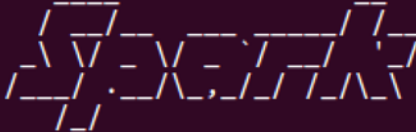
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

Spark context Web UI available at http://10.0.2.15:4040

Spark context available as 'sc' (master = local[4], app id = local-1601889399627).

Spark session available as 'spark'.

Welcome to

 version 3.0.1

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)

Type in expressions to have them evaluated.

Type :help for more information.

```
scala>
```

Adresse et port du spark context Web UI

On peut accéder au
context dans un script
avec la variable `sc`

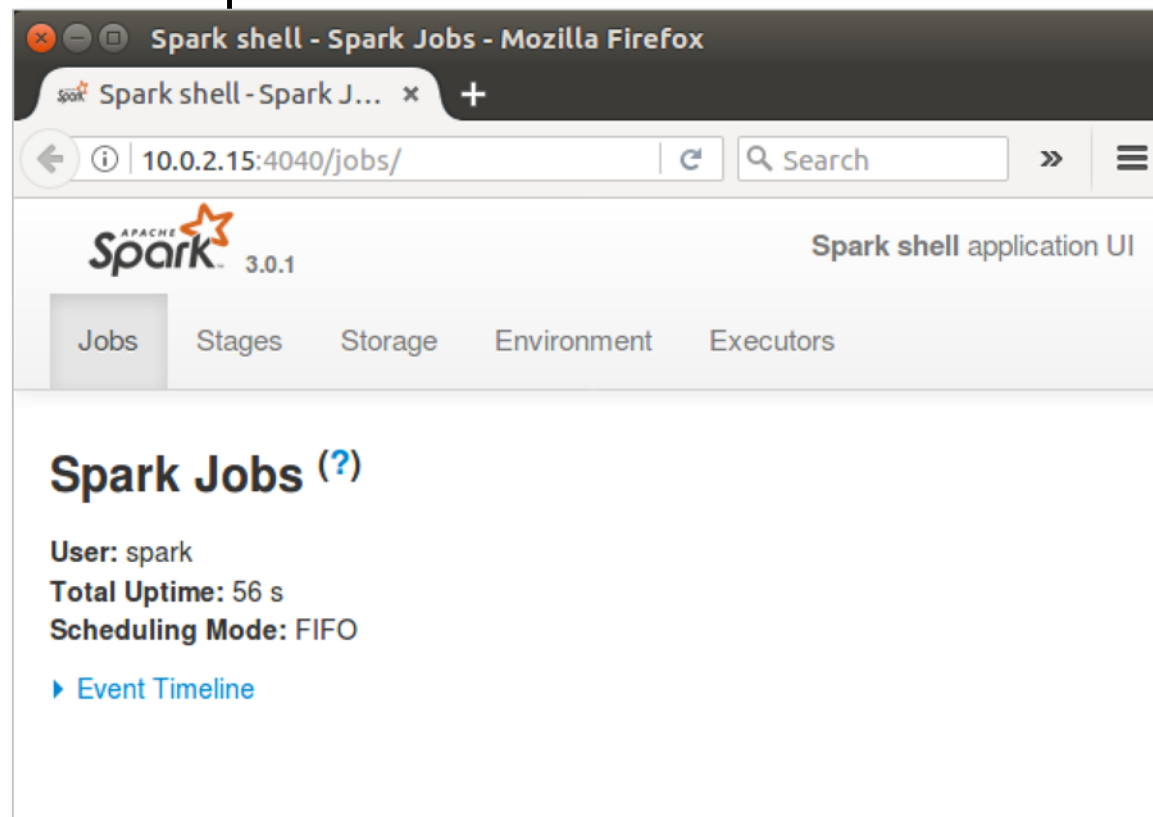
On peut accéder à la session avec la variable `spark`

Lancement en 3 modes

- L'option `--master` permet de préciser à quel type de `cluster manager` l'application Spark peut être envoyée.
- Spark peut fonctionner en se connectant à des `cluster managers` de types différents :
 - `--master spark://HOTE:PORT`: utilise le cluster manager autonome de Spark.
 - `--master mesos://HOTE:PORT`: se connecte à un cluster manager Mesos.
 - `--master yarn`: se connecte à un cluster manager Yarn.
 - `--master local[i]`: pas de cluster manager, Spark fonctionne en mode local. `i` spécifie le nombre d'executors dans le cluster : `local[1]` ou `local[4]`.
- Par défaut, si l'option `--master` n'est pas spécifiée, Spark fonctionne en mode local avec (nb d'executors = nb de cœurs logique de la machine).

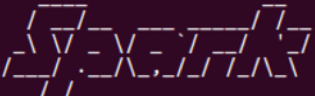
Interface de contrôle Spark

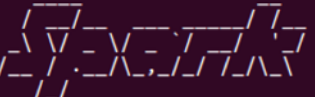
- Spark dispose d'une [interface Web](#) qui permet de consulter les entrailles du système et de mieux comprendre ce qui est fait.
- Elle est accessible sur le port **4040**



Versions de Spark : Scala, Python, Java

- `pyspark` permet de lancer Spark avec un shell python

```
spark@spark-VirtualBox: ~  
spark@spark-VirtualBox:~$ spark-shell --master "local[4]"  
20/10/05 10:16:23 WARN Utils: Your hostname, spark-VirtualBox resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)  
20/10/05 10:16:23 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address  
20/10/05 10:16:24 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable  
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
Spark context Web UI available at http://10.0.2.15:4040  
Spark context available as 'sc' (master = local[4], app id = local-1601889399627).  
Spark session available as 'spark'.  
Welcome to  
 version 3.0.1  
Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)  
Type in expressions to have them evaluated.  
Type :help for more information.  
scala> █
```

```
spark@spark-VirtualBox: ~  
spark@spark-VirtualBox:~$ pyspark  
Python 2.7.12 (default, Nov 19 2016, 06:48:10)  
[GCC 5.4.0 20160609] on linux2  
Type "help", "copyright", "credits" or "license" for more information.  
20/10/05 13:41:48 WARN Utils: Your hostname, spark-VirtualBox resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)  
20/10/05 13:41:48 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address  
20/10/05 13:41:49 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable  
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
20/10/05 13:41:52 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.  
/usr/lib/spark/python/pyspark/context.py:225: DeprecationWarning: Support for Python 2 and Python 3 prior to version 3.6 is deprecated as of Spark 3.0. See also the plan for dropping Python 2 support at https://spark.apache.org/news/plan-for-dropping-python-2-support.html.  
DeprecationWarning)  
Welcome to  
 version 3.0.1  
Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)  
SparkSession available as 'spark'.  
>>> █
```

EXAMPLE : WORD COUNT



Exemple : wordcount avec Scala

- Soit le fichier `input.txt` dans le fichier où vous lancer le `spark-shell`
- Il faut exécuter les 3 commandes Scala

```
/** map */  
var map = sc.textFile("input.txt").flatMap(line => line.split(" ")).map(word => (word,1));  
  
/** reduce */  
var counts = map.reduceByKey(_ + _);  
  
/** save the output to file */  
counts.saveAsTextFile("output/") |
```

On divise chaque ligne avec le séparateur « »

En utilisant la variable Spark context `SC` pour lire le fichier

On mappe à chaque mot un tuple `(word , 1)` 1 nombre d'occurrences

Sauvegarder le résultat dans des fichiers dans le répertoire `output`

Exemple wordcount en Scala

```
spark@spark-VirtualBox: ~/exemples/wordcount

version 3.0.1

Using Scala version 2.12.10 (OpenJDK Server VM, Java 1.8.0_265)
Type in expressions to have them evaluated.
Type :help for more information.

scala> var map = sc.textFile("input.txt").flatMap(line => line.split(" ")).map(w
ord => (word,1));
20/10/05 16:17:02 WARN SizeEstimator: Failed to check whether UseCompressedOops
is set; assuming yes
map: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at map at <co
nsole>:24

scala> var counts = map.reduceByKey(_ + _);
counts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduceByKey
at <console>:25

scala> counts.saveAsTextFile("output/")

scala>
```

```
spark@spark-VirtualBox: ~/exemples/wordcount

spark@spark-VirtualBox:~/exemples/wordcount$ ls output/
part-00000 part-00001 _SUCCESS
spark@spark-VirtualBox:~/exemples/wordcount$
```

Exemple : Spark Context Web UI

10.0.2.15:4040/jobs/ Search Spark shell application UI

Spark Jobs (?)

User: spark
Total Uptime: 35 min
Scheduling Mode: FIFO
Completed Jobs: 1

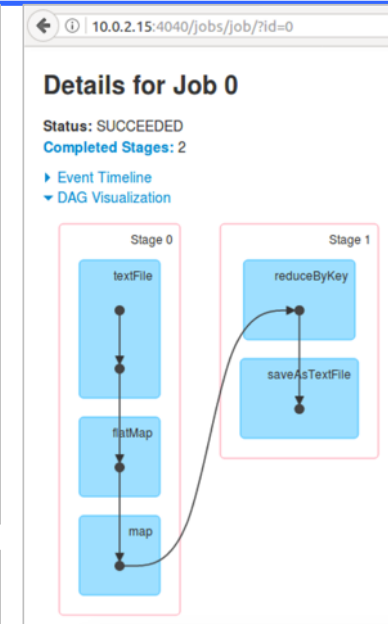
Event Timeline

Completed Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2020/10/05 16:18:06	3 s	2/2	4/4

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go



10.0.2.15:4040/jobs/job/?id=0

Spark shell application UI

Details for Job 0

Status: SUCCEEDED
Completed Stages: 2

Event Timeline

DAG Visualization

Completed Stages (2)

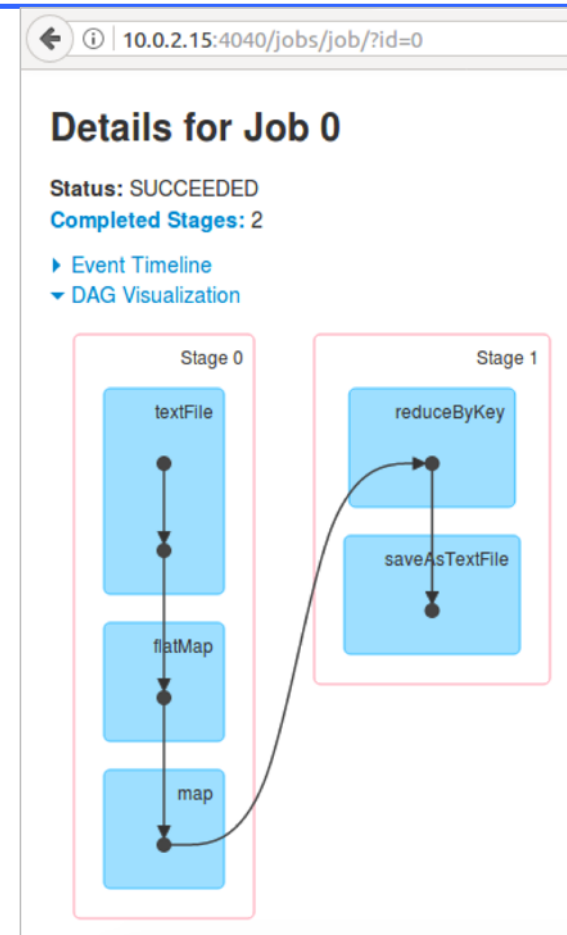
Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	runJob at SparkHadoopWriter.scala:78 +details	2020/10/05 16:18:08	0,7 s	2/2		1167.0 B	1153.0 B	
0	map at <console>:24 +details	2020/10/05 16:18:06	1 s	2/2	1526.0 B			1153.0 B

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Spark Context Web UI - explication

- Il y a 2 étapes (stages),
 - stage0, stage1
- une étape regroupe un ensemble d'opérations qu'il est possible d'exécuter sur une seule machine
- une étape est effectuée en **parallèle** sur les différents fragments des données
- Spark appelle **tâche (task)** l'exécution de l'étape sur un fragment particulier
- la séquence des étapes constitue un **job**.



Exemple : Spark Context Web UI

10.0.2.15:4040/jobs/ Search Spark shell application UI

Spark Jobs (?)

User: spark
Total Uptime: 35 min
Scheduling Mode: FIFO
Completed Jobs: 1

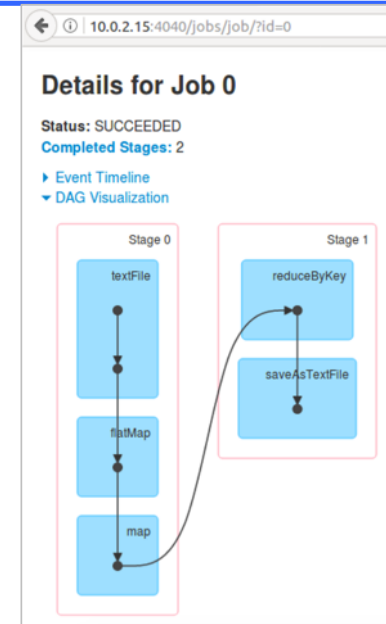
Event Timeline

Completed Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2020/10/05 16:18:06	3 s	2/2	4/4

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go



10.0.2.15:4040/jobs/job/?id=0

Details for Job 0

Status: SUCCEEDED
Completed Stages: 2

Event Timeline

DAG Visualization

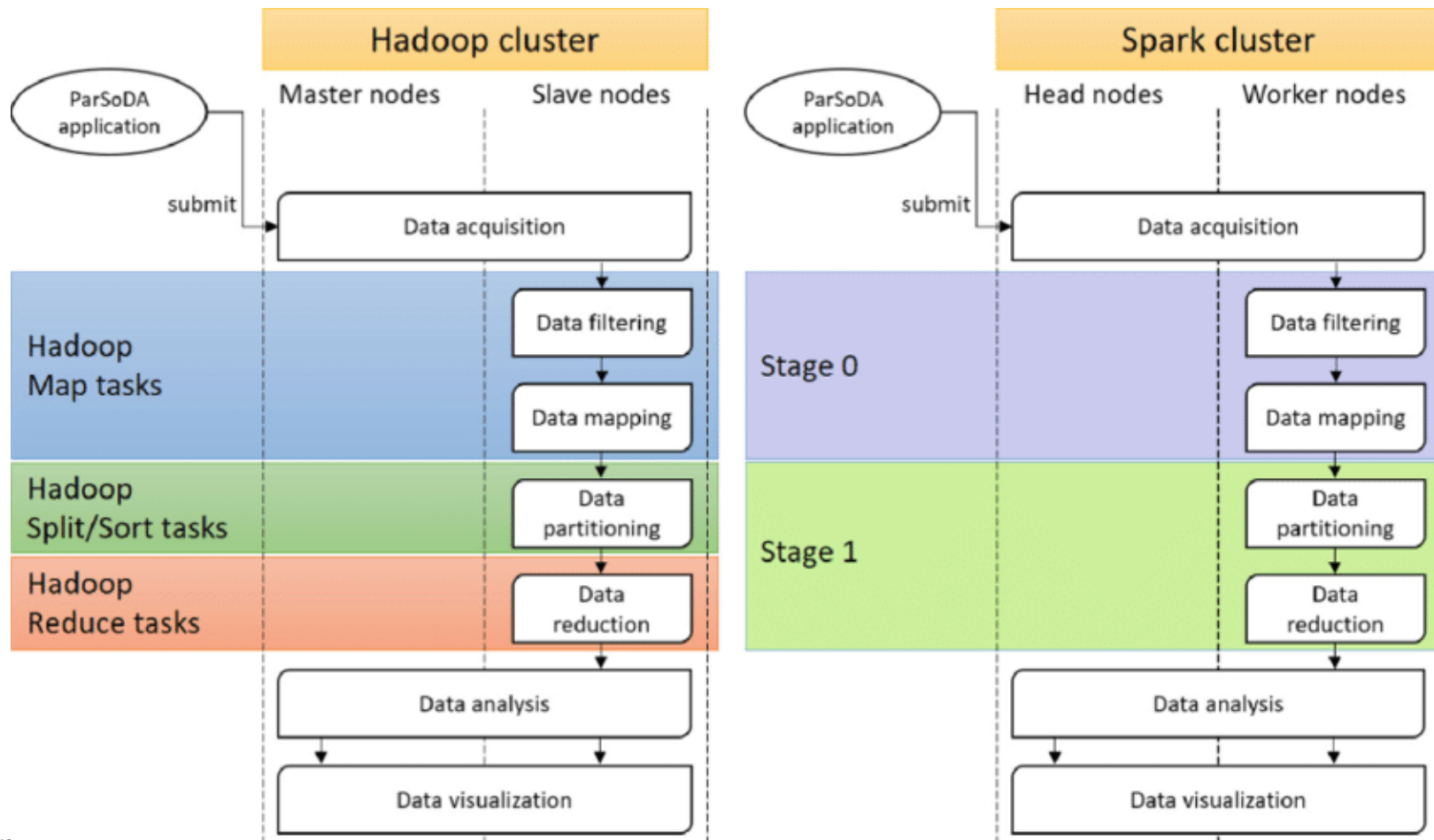
Completed Stages (2)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	runJob at SparkHadoopWriter.scala:78 +details	2020/10/05 16:18:08	0,7 s	2/2		1167.0 B	1153.0 B	
0	map at <console>:24 +details	2020/10/05 16:18:06	1 s	2/2	1526.0 B			1153.0 B

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Spark vs. Hadoop : exécution



EXAMPLE : WORD COUNT - SHELL



Lancement de Spark python

```
spark@spark-VirtualBox: ~/examples/wordcount
spark@spark-VirtualBox:~/examples/wordcount$ pyspark --master local[4]
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
[GCC 5.4.0 20160609] on linux2
Type "help", "copyright", "credits" or "license" for more information.
20/10/05 17:31:54 WARN Utils: Your hostname, spark-VirtualBox resolves to a loop
back address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
20/10/05 17:31:54 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
20/10/05 17:31:57 WARN NativeCodeLoader: Unable to load native-hadoop library fo
r your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
/usr/lib/spark/python/pyspark/context.py:225: DeprecationWarning: Support for Py
thon 2 and Python 3 prior to version 3.6 is deprecated as of Spark 3.0. See also
the plan for dropping Python 2 support at https://spark.apache.org/news/plan-fo
r-dropping-python-2-support.html.
  DeprecationWarning)
Welcome to

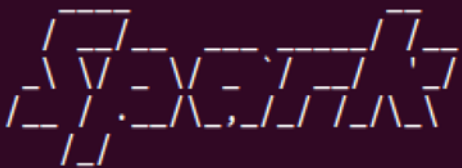
  ____      _
 / ___|  _ \| | | |
 \___ \ |_) | | | |
  ___) | |_) | | | |
 |____|_|_|\___|_|_|_|

version 3.0.1

Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)
SparkSession available as 'spark'.
>>>
```


Exemple : wordcount avec Python

```
input_file = sc.textFile("input.txt")
map = input_file.flatMap(lambda line: line.split(" ")).map(lambda word: (word, 1))
counts = map.reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("output2/")
```



```
version 3.0.1

Using Python version 2.7.12 (default, Nov 19 2016 06:48:10)
SparkSession available as 'spark'.
>>> input_file = sc.textFile("input.txt")
20/10/05 17:55:05 WARN SizeEstimator: Failed to check whether UseCompressedOoops
is set; assuming yes
>>> map = input_file.flatMap(lambda line: line.split(" ")).map(lambda word: (word, 1))
>>> counts = map.reduceByKey(lambda a, b: a + b)
>>> counts.saveAsTextFile("output2/")
>>>
```

```
spark@spark-VirtualBox:~/exemples/wordcount$ ls output2/
part-00000  part-00001  _SUCCESS
```

PySpark : visualisation

```
spark@spark-VirtualBox:~/examples/wordcount$ cat output2/part-00000
(u'', 2)
(u'and', 2)
(u'followed', 1)
(u'is', 3)
(u'reduce', 1)
(u'fault-tolerant', 1)
(u'maintained', 1)
(u'map,', 1)
(u'Spark's", 1)
(u'technology', 1)
```

The screenshot displays the Spark UI interface. The top navigation bar includes tabs for Jobs, Stages, Storage, Environment, Executors, and SQL. The main content area is titled "Spark Jobs (?)" and shows the following details:

- User: spark
- Total Uptime: 28 min
- Scheduling Mode: FIFO
- Completed Jobs: 1

Below this, there is a section for "Completed Jobs (1)" with a table listing the job details:

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	runJob at SparkHadoopWriter.scala:78 runJob at SparkHadoopWriter.scala:78	2020/10/05 17:55:47	6 s	2/2	4/4

At the bottom right, there is a "DAG Visualization" section showing a Directed Acyclic Graph (DAG) for Job 0. The DAG consists of two stages:

- Stage 0: A single task labeled "textFile".
- Stage 1: A sequence of four tasks: "partitionBy", "mapPartitions", "map", and "saveAsTextFile".

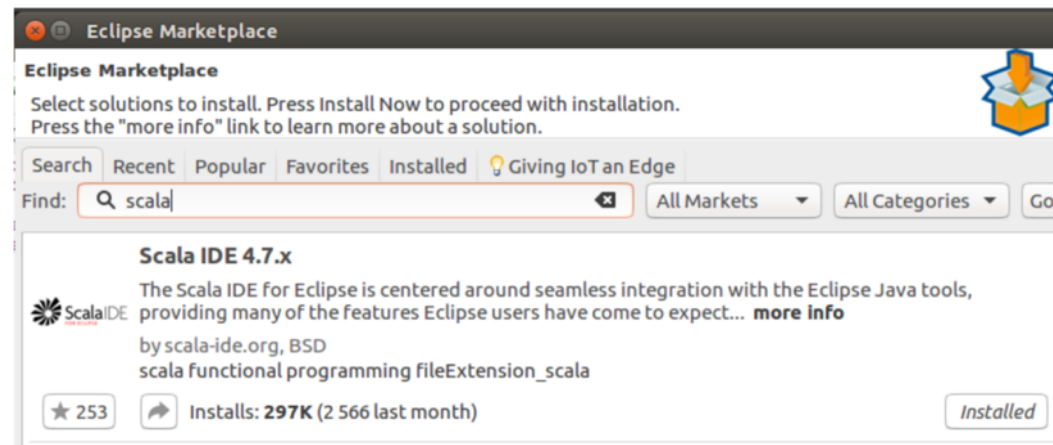
The tasks in Stage 1 are connected sequentially, and the output of Stage 0 is connected to the input of Stage 1.

EXEMPLE : WORD COUNT AVEC APPLICATION

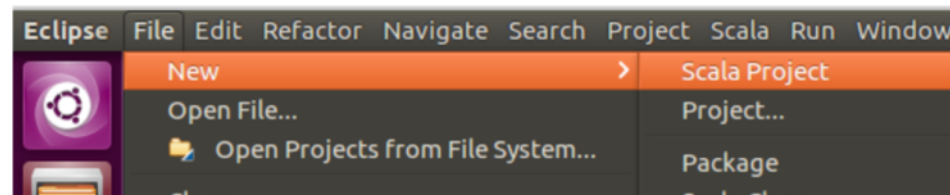


Exemple : wordcount avec application Scala

- Installer Scala IDE (4.7) sur Eclipse (4.7 Oxygen)



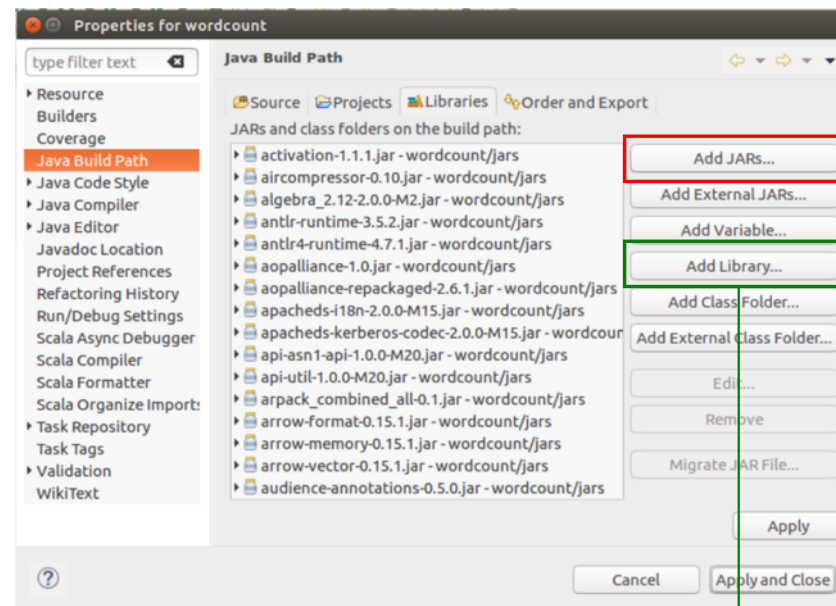
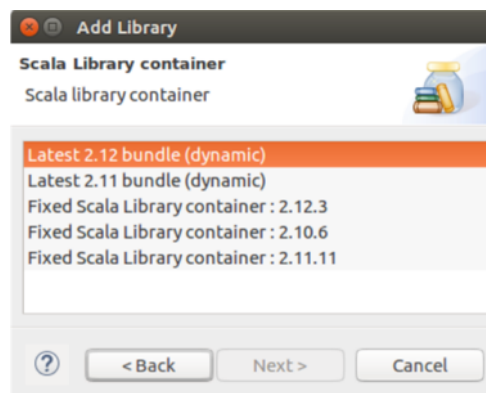
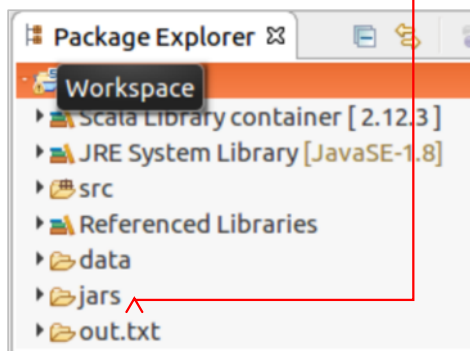
- Créer un projet Scala : [wordcount](#)



Exemple : wordcount avec application Scala

- Ajouter les Jars de Spark au projet

```
spark@spark-VirtualBox:/wordcount/jars/lib$ cd /usr/lib/spark/  
spark@spark-VirtualBox:/usr/lib/spark$ ls  
bin  data  jars  LICENSE  NOTICE  R  RELEASE  yarn  
conf  examples  kubernetes  licenses  python  README.md  sbin
```

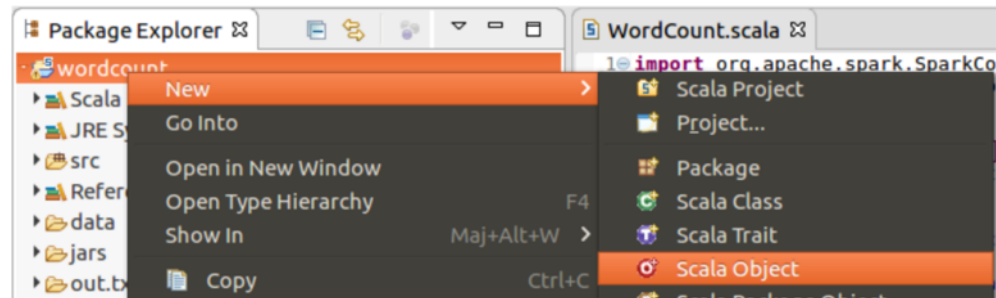


En cas d'erreur

- *changer la version de bibliothèque de scala
- `$sudo mkdir /NOM_PROJET/REP_JAR`
- `$chmod 777 /NOM_PROJET/REP_JAR`

Exemple : wordcount avec application Scala

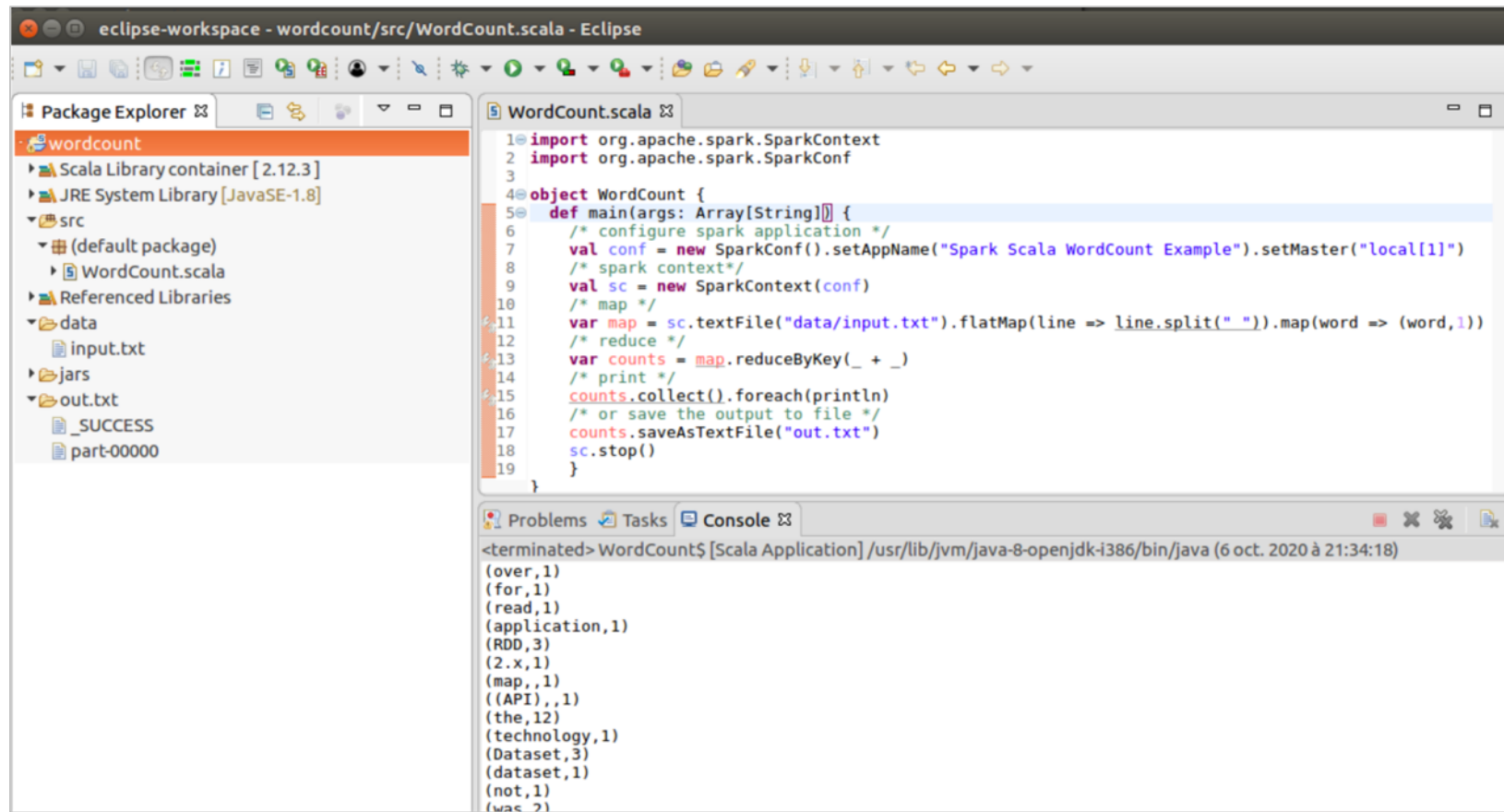
- Créer une nouvelle classe Scala



```
WordCount.scala
1 import org.apache.spark.SparkContext
2 import org.apache.spark.SparkConf
3
4 object WordCount {
5   def main(args: Array[String]) {
6     /* configure spark application */
7     val conf = new SparkConf().setAppName("Spark Scala WordCount Example").setMaster("local[1]")
8     /* spark context */
9     val sc = new SparkContext(conf)
10    /* map */
11    var map = sc.textFile("data/input.txt").flatMap(line => line.split(" ")).map(word => (word, 1))
12    /* reduce */
13    var counts = map.reduceByKey(_ + _)
14    /* print */
15    counts.collect().foreach(println)
16    /* or save the output to file */
17    counts.saveAsTextFile("out.txt")
18    sc.stop()
19  }
20 }
```

Exemple : wordcount avec application Scala

- Exécuter la classe (Run As Scala Application)



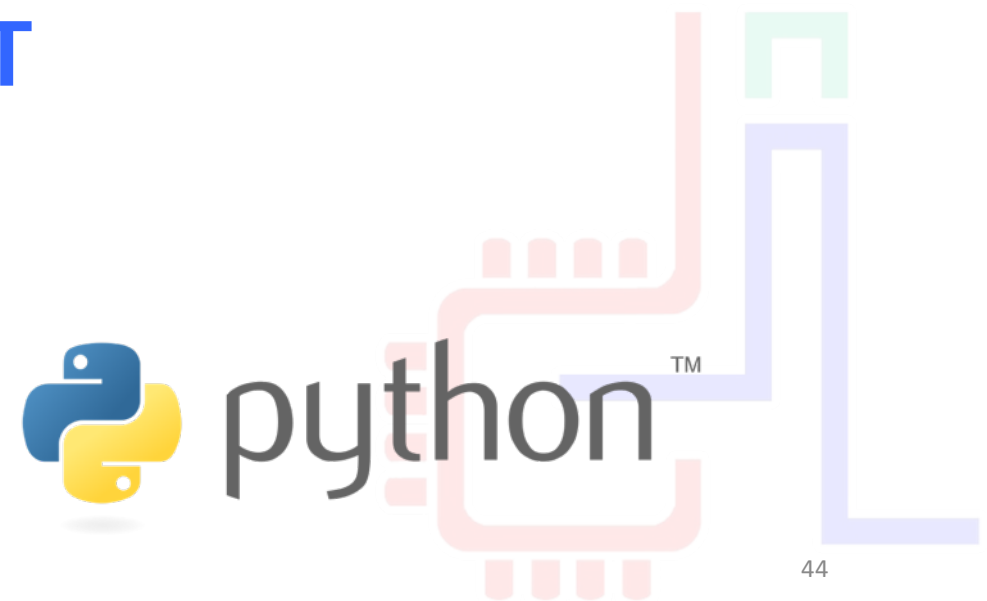
The screenshot shows the Eclipse IDE with a workspace named 'wordcount'. The Package Explorer on the left shows the project structure: 'wordcount' (containing 'Scala Library container [2.12.3]', 'JRE System Library [JavaSE-1.8]', 'src' (with 'WordCount.scala'), 'data' (with 'input.txt'), 'jars', and 'out.txt' (with '_SUCCESS' and 'part-00000')). The main editor displays the 'WordCount.scala' file with the following code:

```
1 import org.apache.spark.SparkContext
2 import org.apache.spark.SparkConf
3
4 object WordCount {
5   def main(args: Array[String]) {
6     /* configure spark application */
7     val conf = new SparkConf().setAppName("Spark Scala WordCount Example").setMaster("local[1]")
8     /* spark context */
9     val sc = new SparkContext(conf)
10    /* map */
11    var map = sc.textFile("data/input.txt").flatMap(line => line.split(" ")).map(word => (word,1))
12    /* reduce */
13    var counts = map.reduceByKey(_ + _)
14    /* print */
15    counts.collect().foreach(println)
16    /* or save the output to file */
17    counts.saveAsTextFile("out.txt")
18    sc.stop()
19  }
20 }
```

The Console at the bottom shows the output of the application, which is a list of words and their counts, sorted by count in descending order:

```
<terminated> WordCount$ [Scala Application] /usr/lib/jvm/java-8-openjdk-i386/bin/java (6 oct. 2020 à 21:34:18)
(over,1)
(for,1)
(read,1)
(application,1)
(RDD,3)
(2.x,1)
(map,,1)
((API),,1)
(the,12)
(technology,1)
(Dataset,3)
(dataset,1)
(not,1)
(was,2)
```

EXEMPLE : WORD COUNT AVEC APPLICATION



Exemple : wordcount – Application python

- Ecrire un programme python

```
import sys

from pyspark import SparkContext, SparkConf

if __name__ == "__main__":
    # create Spark context with Spark configuration
    conf = SparkConf().setAppName("Word Count - Python").set(
        ("spark.hadoop.yarn.resourcemanager.address", "10.0.2.15:8032")
    )
    sc = SparkContext(conf=conf)

    # read in text file and split each document into words
    words = sc.textFile("/home/spark/examples/wordcount/input.txt").flatMap(lambda line:
line.split(" "))

    # count the occurrence of each word
    wordCounts = words.map(lambda word: (word, 1)).reduceByKey(lambda a,b:a +b)

    wordCounts.saveAsTextFile("/home/spark/examples/wordcount/python-app/output/") |
```

- Exécuter la commande `$ spark-submit wordcount.py`

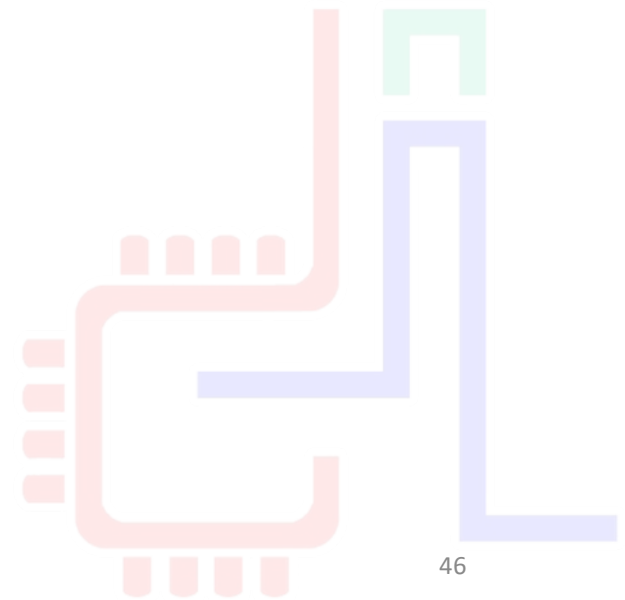
- Résultat

```
spark@spark-VirtualBox:~/examples/wordcount/python-app$ ls -l output/
total 4
-rw-r--r-- 1 spark spark 1571 6 17:39 part-00000
-rw-r--r-- 1 spark spark 0 6 17:39 _SUCCESS
```

CONFIGURATION DE SPARK

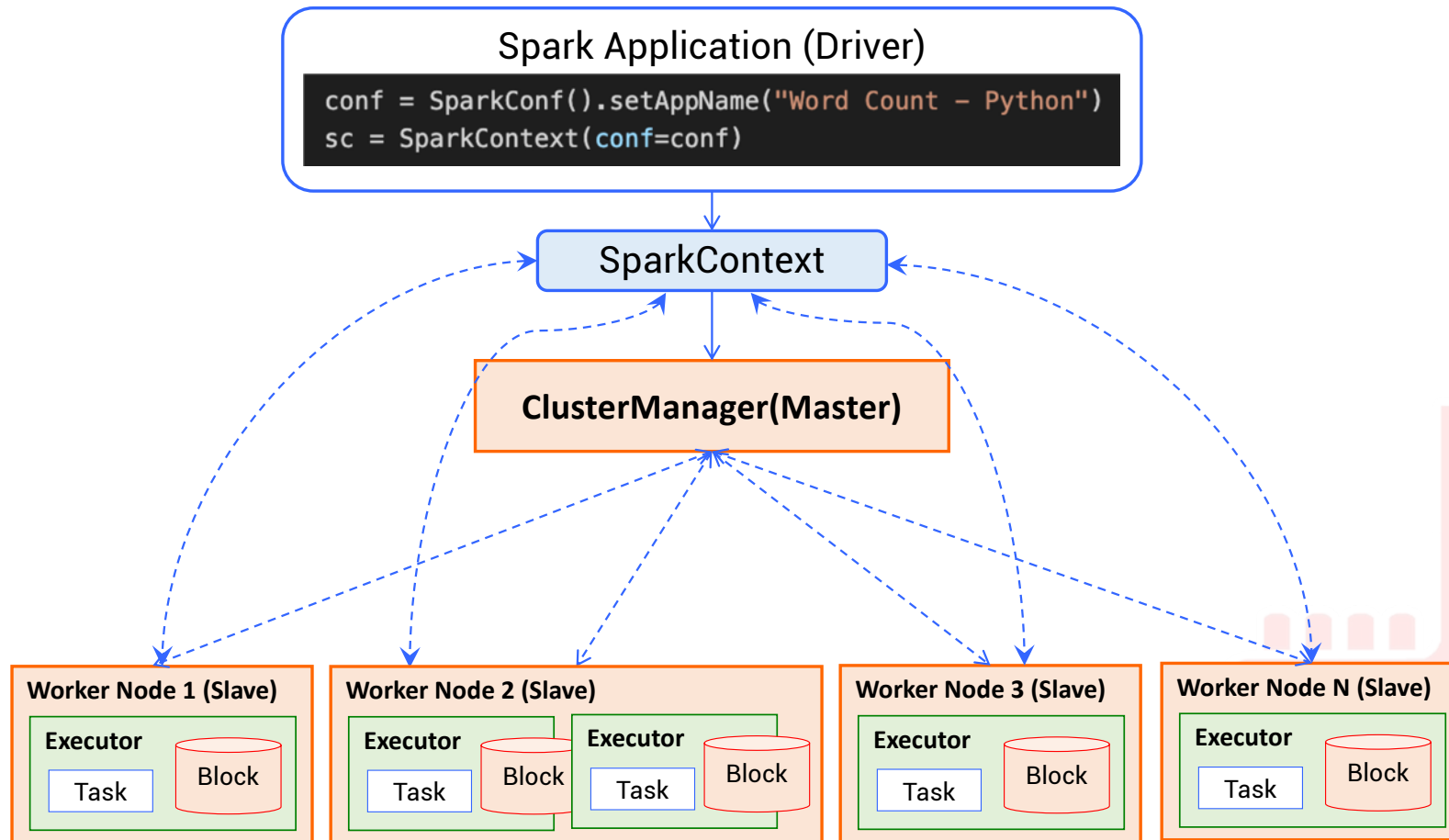
2K20

TBM



46

Batch Processing



Configurer un Spark Cluster

■ Configurer Spark Master Node

- Aller au répertoire de configuration de spark `SPARK_HOME/conf/`
- Editer le fichier `spark-env.sh` (à Créer s'il n'existe pas en copiant `spark-env.sh.template`)

```
spark@spark-VirtualBox: /usr/lib/spark/conf
spark@spark-VirtualBox:~$ cd /usr/lib/spark/conf
spark@spark-VirtualBox:/usr/lib/spark/conf$ ls
fairscheduler.xml.template  slaves.template
log4j.properties.template  spark-defaults.conf.template
metrics.properties.template spark-env.sh.template
spark@spark-VirtualBox:/usr/lib/spark/conf$ cp spark-env.sh.template spark-env.sh
```

- Ajouter la variable `SPARK_MASTER_HOST`

```
spark-env.sh (/usr/lib/spark/conf) - gedit
Ouvrir
# Options for the daemons used in the standalone deploy mode
# - SPARK_MASTER_HOST, to bind the master to a different IP address or hostname
export SPARK_MASTER_HOST='127.0.0.1'
# - SPARK_MASTER_PORT / SPARK_MASTER_WEBUI_PORT, to use non-default ports for the master
```

Configurer un Spark Cluster

- Lancer spark comme Master en lançant la commande
`SPARK_HOME/sbin/start-master.sh`

```
spark@spark-VirtualBox: /usr/lib/spark/sbin
spark@spark-VirtualBox:/usr/lib/spark$ cd sbin
spark@spark-VirtualBox:/usr/lib/spark/sbin$ ./start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /usr/lib/spark/logs/spark-s
park-org.apache.spark.deploy.master.Master-1-spark-VirtualBox.out
spark@spark-VirtualBox:/usr/lib/spark/sbin$
```

- Output du fichier log `.out`

```
Spark Command: /usr/lib/jvm/default-java/jre/bin/java -cp /usr/lib/spark/conf:/usr/lib/spark/jars/
* -Xmx1g org.apache.spark.deploy.master.Master --host 127.0.0.1 --port 7077 --webui-port 8080
=====
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
20/10/11 13:38:19 INFO Master: Started daemon with process name: 4289@spark-VirtualBox
20/10/11 13:38:19 INFO SignalUtils: Registered signal handler for TERM
20/10/11 13:38:19 INFO SignalUtils: Registered signal handler for HUP
```

- Lancer Spark Slave (Worker) Node

- Lancer un Worker : `$ ./start-slave.sh spark://<master.ip.address>:7077`

```
spark@spark-VirtualBox:/usr/lib/spark/sbin$ ./start-slave.sh spark://127.0.0.1:7077
starting org.apache.spark.deploy.worker.Worker, logging to /usr/lib/spark/logs/spark-spark-org.apac
he.spark.deploy.worker.Worker-1-spark-VirtualBox.out
```

Configurer un Spark Cluster

- Output du log Worker

```
spark@spark-VirtualBox: /usr/lib/spark/sbin
Spark Command: /usr/lib/jvm/default-java/jre/bin/java -cp /usr/lib/spark/conf/:/usr/lib/spark/jars/
* -Xmx1g org.apache.spark.deploy.worker.Worker --webui-port 8081 spark://127.0.0.1:7077
=====
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
20/10/11 13:41:38 INFO Worker: Started daemon with process name: 4366@spark-VirtualBox
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for TERM
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for HUP
20/10/11 13:41:38 INFO SignalUtils: Registered signal handler for INT
20/10/11 13:41:38 WARN Utils: Your hostname, spark-VirtualBox resolves to a loopback address: 127.0
.1.1; using 10.0.2.15 instead (on interface enp0s3)
20/10/11 13:41:38 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
20/10/11 13:41:40 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
20/10/11 13:41:40 INFO SecurityManager: Changing view acls to: spark
20/10/11 13:41:40 INFO SecurityManager: Changing modify acls to: spark
20/10/11 13:41:40 INFO SecurityManager: Changing view acls groups to:
20/10/11 13:41:40 INFO SecurityManager: Changing modify acls groups to:
20/10/11 13:41:40 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled;
users with view permissions: Set(spark); groups with view permissions: Set(); users with modify
permissions: Set(spark); groups with modify permissions: Set()
20/10/11 13:41:41 INFO Utils: Successfully started service 'sparkWorker' on port 45217.
20/10/11 13:41:41 INFO Worker: Starting Spark worker 10.0.2.15:45217 with 1 cores, 3.5 GiB RAM
20/10/11 13:41:41 INFO Worker: Running Spark version 3.0.1
20/10/11 13:41:41 INFO Worker: Spark home: /usr/lib/spark
20/10/11 13:41:41 INFO ResourceUtils: =====
=
20/10/11 13:41:41 INFO ResourceUtils: Resources for spark.worker:
20/10/11 13:41:41 INFO ResourceUtils: =====
=
20/10/11 13:41:42 INFO Utils: Successfully started service 'WorkerUI' on port 8081.
20/10/11 13:41:42 INFO WorkerWebUI: Bound WorkerWebUI to 0.0.0.0, and started at http://10.0.2.15:8
081
20/10/11 13:41:42 INFO Worker: Connecting to master 127.0.0.1:7077...
```

Configurer un Spark Cluster

- **Configurer Spark Slave (Worker) Node**

- Aller au répertoire de configuration de spark [SPARK_HOME/conf/](#)
- Editer le fichier [slaves](#) (à Créer s'il n'existe pas en copiant `slaves.template`)

```
# A Spark Worker will be started on each of the machines listed below.  
localhost  
localhost
```

